

Backlog del Sprint

Plataforma de Alquiler de Autos — Proyecto Universitario Versión 2 — Arquitectura de un solo servicio

Campo	Valor
Equipo	Arango, Corrales, Cifuentes, Suarez, Cardona, Murcia, Pedroza
Stack	Java + Spring Boot + PostgreSQL
Metodología	SCRUM
Entrega	Miércoles
Total HU	21 historias — 3 por integrante
Arquitectura	1 solo servicio (monolito por capas) — artefacto rentacars, paquete raíz com.rentacars

Cambios respecto a la versión anterior

- Se elimina la división en **4 microservicios** (ms-tiendas, ms-catalogo, ms-clientes, ms-alquileres). Todo vive en un único proyecto Spring Boot.
- **Desaparecen los FeignClient**. Donde antes había una llamada HTTP entre servicios, ahora hay una **inyección directa de dependencia** entre services (por ejemplo, AlquilerServiceImpl inyecta AutoService).
- Se añade la **capa** service, que faltaba en la estructura de paquetes creada por el equipo: interfaz en com.rentacars.service e implementación en com.rentacars.service.impl.
- Se añaden los paquetes exception (manejo centralizado de 404 y 400) y mapper (conversión entidad ↔ DTO).
- Se ajusta la **nomenclatura de DTOs** a la convención ya adoptada en el repositorio: CreateTiendaRequest, UpdateTiendaRequest, CreateTiendaResponse (en vez de TiendaRequestDTO).
- Las HU y su reparto **no cambian**: siguen siendo las mismas 21 historias, 3 por integrante. Solo cambia la capa técnica donde se implementan.

Estructura de paquetes del proyecto

Base: la estructura ya creada por el equipo. En **verde** lo que se añade en esta versión.

```

com.rentacars
|-- controller
|   |-- AlquilerController.java           |-- ClienteController.java
|   |-- AutoController.java              |-- Detalle_autoController.java
|   |-- CategoriaController.java         \-- TiendaController.java
|-- dto
|   |-- request
|   |   |-- CreateAlquilerRequest.java   |-- UpdateAlquilerRequest.java
|   |   |-- CreateAutoRequest.java       |-- UpdateAutoRequest.java
|   |   |-- ... (6 entidades)             \-- ... (6 entidades)
|   \-- response
|       |-- CreateAlquilerResponse.java
|       |-- CreateAutoResponse.java
|       \-- ... (6 entidades)
|-- model
|   |-- Alquiler.java                   |-- Categoria.java           |-- Detalle_auto.java
|   |-- Auto.java                       |-- Cliente.java             \-- Tienda.java
|-- repository
|   |-- AlquilerRepository.java          |-- ClienteRepository.java
|   |-- AutoRepository.java              |-- Detalle_autoRepository.java
|   |-- CategoriaRepository.java         \-- TiendaRepository.java
\-- RentacarsApplication.java

```

```

com.rentacars
|-- service                                <-- NUEVO
|   |-- AlquilerService.java              (interfaz)
|   |-- AutoService.java                  (interfaz)
|   |-- CategoriaService.java              (interfaz)
|   |-- ClienteService.java                (interfaz)
|   |-- Detalle_autoService.java           (interfaz)
|   |-- TiendaService.java                 (interfaz)
|   \-- impl
|       |-- AlquilerServiceImpl.java       |-- ClienteServiceImpl.java
|       |-- AutoServiceImpl.java           |-- Detalle_autoServiceImpl.java
|       |-- CategoriaServiceImpl.java       \-- TiendaServiceImpl.java
|-- exception                             <-- NUEVO
|   |-- GlobalExceptionHandler.java        (@RestControllerAdvice)
|   |-- ResourceNotFoundException.java      (-> 404 Not Found)
|   \-- BadRequestException.java           (-> 400 Bad Request)
\-- mapper                                <-- NUEVO
    |-- AlquilerMapper.java                |-- ClienteMapper.java
    |-- AutoMapper.java                    |-- Detalle_autoMapper.java
    |-- CategoriaMapper.java                \-- TiendaMapper.java

```

Resumen de asignación

Ya no hay columna de microservicio. Cada integrante trabaja sobre un **dominio** dentro del mismo proyecto; el dominio indica qué entidades y qué clases toca principalmente.

Integrante	HUs asignadas	Dominio / clases principales
Arango	HU-01, HU-02, HU-03	Tiendas — Tienda*
Corrales	HU-04, HU-05, HU-24	Tiendas + Alquileres — Tienda*, Alquiler*
Cifuentes	HU-06, HU-07, HU-08	Catálogo — Categoria*, Auto*, Detalle_auto*
Suarez	HU-09, HU-10, HU-11	Catálogo — Auto*, Detalle_auto*
Cardona	HU-12, HU-13, HU-22	Catálogo + Alquileres — Auto*, Alquiler*
Murcia	HU-14, HU-15, HU-16	Cientes — Cliente*
Pedroza	HU-18, HU-20, HU-21	Alquileres — Alquiler*

Reglas de trabajo con un solo repositorio

- Al ser un solo proyecto, **varias personas tocarán el mismo archivo** (ej. AutoController lo tocan Cifuentes, Suarez y Cardona). Cada quien añade **solo su método**, no reescribe los ajenos.
- Ramas: feature/<apellido>-HU-<número> (ej. feature/pedroza-HU-18), todas salen de develop.
- Antes de abrir PR: git pull origin develop y resolver conflictos en local.
- Las **entidades y los repositorios se crean una sola vez**. Quien tenga la HU de creación de esa entidad (HU-01 Tienda, HU-06 Categoria, HU-08 Auto y Detalle_auto, HU-14 Cliente, HU-18 Alquiler) la crea primero y avisa al grupo.
- El GlobalExceptionHandler y las excepciones lo monta **una sola persona al inicio del sprint** (sugerido: Arango, junto con HU-01); el resto solo lanza ResourceNotFoundException o BadRequestException.

Arango

HU-01, HU-02, HU-03 — Dominio Tiendas

HU-01 Registrar tienda

Como administrador, quiero registrar una nueva tienda, para ampliar la red de puntos de alquiler.

Endpoint: POST /tiendas

Body de entrada (JSON):

```
{ "nombre": "Tienda Norte", "ciudad": "Bogota", "direccion": "Calle 100 #15-20" }
```

Respuesta exitosa:

```
{ "id_tienda": 1, "nombre": "Tienda Norte", "ciudad": "Bogota",  
  "direccion": "Calle 100 #15-20" }
```

Reglas de negocio:

- Todos los campos son obligatorios.
- Si falta alguno → **400 Bad Request**.

Capas a implementar:

- Tienda.java → entidad JPA (tabla *tiendas*)
- TiendaRepository.java → extiende JpaRepository
- CreateTiendaRequest.java / CreateTiendaResponse.java → DTOs con @NotBlank
- TiendaMapper.java → toEntity() y toCreateResponse()
- **TiendaService.java** → interfaz, método crearTienda(CreateTiendaRequest)
- **TiendaServiceImpl.java** → @Service, lógica de creación
- TiendaController.java → endpoint POST, inyecta TiendaService (la interfaz, no la impl)

Tarea transversal de esta HU (hacerla primero)

- Crear también el paquete exception completo: GlobalExceptionHandler, ResourceNotFoundException y BadRequestException. Todo el equipo depende de esto.

HU-02 Actualizar tienda

Como administrador, quiero actualizar los datos de una tienda, para mantener la información correcta.

Endpoint: PUT /tiendas/{id}

Body de entrada (JSON):

```
{ "nombre": "Tienda Norte Actualizada", "ciudad": "Medellin",  
  "direccion": "Carrera 50 #30-10" }
```

Respuesta exitosa:

```
{ "id_tienda": 1, "nombre": "Tienda Norte Actualizada", "ciudad": "Medellin",  
  "direccion": "Carrera 50 #30-10" }
```

Reglas de negocio:

- Si la tienda no existe → **404 Not Found**.
- Actualizar solo los campos que lleguen en el body.

Capas a implementar:

- UpdateTiendaRequest.java → campos opcionales (sin @NotBlank)
- **TiendaService.java** → actualizarTienda(Long id, UpdateTiendaRequest)

- **TiendaServiceImpl.java** → validar existencia (lanzar `ResourceNotFoundException`) y actualizar campo a campo
- `TiendaRepository.java` → `findById()` y `save()`
- `TiendaController.java` → endpoint PUT

HU-03 Listar tiendas por ciudad

Como cliente, quiero listar tiendas por ciudad, para saber dónde puedo retirar un auto.

Endpoint: GET /tiendas?ciudad=Bogota

Respuesta exitosa:

```
[{ "id_tienda": 1, "nombre": "Tienda Norte", "ciudad": "Bogota",
  "direccion": "Calle 100 #15-20" }]
```

Reglas de negocio:

- El parámetro ciudad es opcional.
- Si no se envía → retornar todas las tiendas.
- Si se envía → filtrar ignorando mayúsculas/minúsculas.

Capas a implementar:

- **TiendaService.java** → `listarTiendas(String ciudad)`
- **TiendaServiceImpl.java** → lógica condicional (`findAll()` vs. filtro)
- `TiendaRepository.java` → `findByCiudadIgnoreCase(String ciudad)`
- `TiendaController.java` → endpoint GET con `@RequestParam(required = false)`

Corrales

HU-04, HU-05, HU-24 — Dominio Tiendas + Alquileres

HU-04 Eliminar tienda

Como administrador, quiero eliminar una tienda, para retirar puntos que ya no operan.

Endpoint: DELETE /tiendas/{id}

Respuesta exitosa:

```
204 No Content (sin body)
```

Reglas de negocio:

- Validar que la tienda exista antes de eliminar.
- Si no existe → **404 Not Found**.

Capas a implementar:

- **TiendaService.java** → eliminarTienda(Long id)
- **TiendaServiceImpl.java** → validar existencia y eliminar
- **TiendaRepository.java** → existsById() y deleteById()
- **TiendaController.java** → endpoint DELETE, retorna ResponseEntity.noContent()

HU-05 Consultar tienda por ID

Como sistema, quiero consultar una tienda por ID, para vincularla con los autos disponibles.

Endpoint: GET /tiendas/{id}

Respuesta exitosa:

```
{ "id_tienda": 1, "nombre": "Tienda Norte", "ciudad": "Bogota",  
  "direccion": "Calle 100 #15-20" }
```

Reglas de negocio:

- Si no existe → **404 Not Found**.
- **Cambio v2:** este método ya no lo consumen otros microservicios por HTTP. Ahora AutoServiceImpl inyecta TiendaService y llama al método directamente.

Capas a implementar:

- **TiendaService.java** → obtenerTienda(Long id)
- **TiendaServiceImpl.java** → buscar y retornar, o lanzar ResourceNotFoundException
- **TiendaRepository.java** → findById()
- **TiendaController.java** → endpoint GET por ID

HU-24 Registrar devolución de auto

Como administrador, quiero registrar la devolución de un auto, para cerrar el alquiler y actualizar disponibilidad.

Endpoint: PUT /alquileres/{id}/devolucion

Respuesta exitosa:

```
{ "id_alquiler": 1, "estado": "CERRADO", "fecha_inicio": "2026-08-07",  
  "fecha_fin": "2026-08-10", "precio_total": 150000 }
```

Reglas de negocio:

- Validar que el alquiler exista → si no, **404 Not Found**.

- Marcar el alquiler como CERRADO.
- **Cambio v2:** ya no se usa CatalogoFeignClient. AlquilerServiceImpl inyecta AutoService y llama a actualizarDisponibilidad(idAuto, true).

Capas a implementar:

- **AlquilerService.java** → registrarDevolucion(Long id)
- **AlquilerServiceImpl.java** → lógica de cierre + llamada a autoService.actualizarDisponibilidad()
- AlquilerRepository.java → findById() y save()
- AlquilerController.java → endpoint PUT /alquileres/{id}/devolucion

Ejemplo de la inyección que reemplaza al FeignClient

```
• @Service
  @RequiredArgsConstructor
  public class AlquilerServiceImpl implements AlquilerService {
    private final AlquilerRepository alquilerRepository;
    private final AutoService autoService; // antes: CatalogoFeignClient
  }
```

Cifuentes

HU-06, HU-07, HU-08 — Dominio Catálogo

HU-06 Registrar categoría

Como administrador, quiero registrar una categoría de auto, para clasificar el inventario.

Endpoint: POST /categorias

Body de entrada (JSON):

```
{ "nombre": "SUV", "descripcion": "Vehiculos todo terreno de alto cilindraje" }
```

Respuesta exitosa:

```
{ "id_categoria": 1, "nombre": "SUV",  
  "descripcion": "Vehiculos todo terreno de alto cilindraje" }
```

Reglas de negocio:

- Todos los campos son obligatorios → si falta alguno, **400 Bad Request**.
- No se pueden registrar dos categorías con el mismo nombre.

Capas a implementar:

- Categoria.java → entidad JPA (tabla *categorias*)
- CategoriaRepository.java → existsByNombre(String nombre)
- CreateCategoriaRequest.java / CreateCategoriaResponse.java
- CategoriaMapper.java → entidad ↔ DTO
- **CategoriaService.java** → crearCategoria(CreateCategoriaRequest)
- **CategoriaServiceImpl.java** → validación de nombre duplicado (lanza BadRequestException)
- CategoriaController.java → endpoint POST

HU-07 Listar categorías

Como administrador, quiero listar todas las categorías, para gestionar la clasificación del catálogo.

Endpoint: GET /categorias

Respuesta exitosa:

```
[{ "id_categoria": 1, "nombre": "SUV",  
  "descripcion": "Vehiculos todo terreno..." }]
```

Reglas de negocio:

- Si no hay categorías → retornar lista vacía, no un error.

Capas a implementar:

- **CategoriaService.java** → listarCategorias()
- **CategoriaServiceImpl.java** → findAll() + mapeo a lista de response
- CategoriaRepository.java → findAll()
- CategoriaController.java → endpoint GET

HU-08 Registrar auto con detalles

Como administrador, quiero registrar un auto con todos sus detalles, para agregarlo al inventario.

Endpoint: POST /autos

Body de entrada (JSON):


```
{ "modelo": "Tucson", "marca": "Hyundai", "anio": "2023", "placa": "ABC123",  
  "precio_dia": 150000, "oferta_porcentaje": 10,  
  "imagen": "https://url.com/img.jpg", "id_tienda": 1, "id_categoria": 1 }
```

Respuesta exitosa:

```
{ "id_auto": 1, "disponibilidad": true, "id_tienda": 1, "id_categoria": 1,  
  "detalles": { "modelo": "Tucson", "precio_dia": 150000,  
    "oferta_porcentaje": 10 } }
```

Reglas de negocio:

- Al crear el auto, disponibilidad inicia en true automáticamente.
- Todos los campos son obligatorios.
- Guardar primero en *autos*, luego en *detalles_autos* usando el *id_auto* generado.
- **Cambio v2:** validar que la tienda y la categoría existan inyectando *TiendaService* y *CategoriaService* (antes hubiera sido una llamada HTTP).

Capas a implementar:

- *Auto.java* y *Detalle_auto.java* → dos entidades JPA
- *AutoRepository.java* y *Detalle_autoRepository.java*
- *CreateAutoRequest.java* / *CreateAutoResponse.java*, *CreateDetalle_autoRequest.java*
- *AutoMapper.java* y *Detalle_autoMapper.java*
- **AutoService.java** → *crearAuto(CreateAutoRequest)*
- **AutoServiceImpl.java** → lógica de creación en dos tablas, anotado con *@Transactional*
- *AutoController.java* → endpoint POST

Ventaja de la arquitectura de un solo servicio

- Al estar *autos* y *detalles_autos* en la misma base y el mismo proceso, basta con anotar el método con *@Transactional*: si falla el guardado del detalle, se revierte también el auto. Con microservicios esto habría requerido compensación manual.

HU-09 Buscar autos disponibles

Como cliente, quiero buscar autos disponibles filtrando por ciudad o categoría, para encontrar el que necesito.

Endpoint: GET /autos?ciudad=Bogota&id_categoria=1

Respuesta exitosa:

```
[{ "id_auto": 1, "modelo": "Tucson", "marca": "Hyundai", "precio_dia": 150000,
  "oferta_porcentaje": 10, "disponibilidad": true }]
```

Reglas de negocio:

- Solo retornar autos con disponibilidad = true.
- Los filtros ciudad e id_categoria son opcionales.
- Si no se envía ningún filtro → retornar todos los autos disponibles.
- **Cambio v2:** el filtro por ciudad ahora se resuelve con un JOIN a la tabla *tiendas* en la misma query, no con una llamada a ms-tiendas.

Capas a implementar:

- **AutoService.java** → buscarAutos(String ciudad, Long idCategoria)
- **AutoServiceImpl.java** → lógica condicional sobre los parámetros
- **AutoRepository.java** → @Query con JOIN a tiendas, filtrando por disponibilidad y parámetros opcionales
- **AutoController.java** → endpoint GET con @RequestParam(required = false)

HU-10 Actualizar detalles de auto

Como administrador, quiero actualizar los detalles de un auto, para mantener el catálogo al día.

Endpoint: PUT /autos/{id}

Body de entrada (JSON):

```
{ "precio_dia": 160000, "oferta_porcentaje": 15,
  "imagen": "https://nueva-imagen.com/tucson.jpg" }
```

Respuesta exitosa:

```
{ "id_auto": 1, "modelo": "Tucson", "precio_dia": 160000,
  "oferta_porcentaje": 15, "imagen": "https://nueva-imagen.com/tucson.jpg" }
```

Reglas de negocio:

- Si el auto no existe → **404 Not Found**.
- Solo actualizar los campos que lleguen en el body.
- Actualizar en la tabla *detalles_autos*.

Capas a implementar:

- **UpdateDetalle_autoRequest.java** → campos opcionales
- **AutoService.java** → actualizarDetalles(Long id, UpdateDetalle_autoRequest)
- **AutoServiceImpl.java** → validar existencia y actualizar campo a campo
- **Detalle_autoRepository.java** → findByIdAuto() y save()
- **AutoController.java** → endpoint PUT

HU-11 Actualizar disponibilidad de auto

Como sistema, quiero actualizar la disponibilidad de un auto, para reflejar si está alquilado o libre.

Endpoint: PATCH /autos/{id}/disponibilidad

Body de entrada (JSON):

```
{ "disponibilidad": false }
```

Respuesta exitosa:

```
{ "id_auto": 1, "disponibilidad": false }
```

Reglas de negocio:

- Si el auto no existe → **404 Not Found**.
- Solo actualiza el campo disponibilidad en la tabla *autos*.
- **Cambio v2:** el endpoint PATCH se mantiene para uso administrativo, pero **el consumo interno ya no pasa por HTTP**: AlquilerServiceImpl llama directamente a autoService.actualizarDisponibilidad(id, valor).

Capas a implementar:

- **AutoService.java** → actualizarDisponibilidad(Long id, boolean disponibilidad)
- **AutoServiceImpl.java** → validar existencia y actualizar
- **AutoRepository.java** → findById() y save()
- **AutoController.java** → endpoint PATCH

Coordinación con Pedroza y Cardona

- Este es el método más consumido del proyecto: lo usan HU-18 (crear alquiler), HU-22 (cancelar) y HU-24 (devolución). Debe estar listo y mergeado a develop **antes** de que empiecen esas tres HU.

HU-12 Ver detalle completo de un auto

Como cliente, quiero ver el detalle completo de un auto con precio calculado, para decidir si lo alquilo.

Endpoint: GET /autos/{id}

Respuesta exitosa:

```
{ "id_auto": 1, "modelo": "Tucson", "marca": "Hyundai", "anio": "2023",  
  "placa": "ABC123", "precio_dia": 150000, "oferta_porcentaje": 10,  
  "precio_con_oferta": 135000, "disponibilidad": true }
```

Reglas de negocio:

- Si no existe → **404 Not Found**.
- Calcular: $\text{precio_con_oferta} = \text{precio_dia} - (\text{precio_dia} * \text{oferta_porcentaje} / 100)$.
- Retornar datos de *autos* y *detalles_autos* combinados.

Capas a implementar:

- **AutoService.java** → obtenerDetalle(Long id)
- **AutoServiceImpl.java** → lógica del cálculo de precio con oferta
- **AutoMapper.java** → combinar las dos entidades en un solo response
- **AutoRepository.java** y **Detalle_autoRepository.java** → consultas por ID
- **AutoController.java** → endpoint GET por ID

HU-13 Eliminar auto

Como administrador, quiero eliminar un auto del catálogo, para retirar vehículos fuera de operación.

Endpoint: DELETE /autos/{id}

Respuesta exitosa:

```
204 No Content
```

Reglas de negocio:

- Si no existe → **404 Not Found**.
- Eliminar primero en *detalles_autos*, luego en *autos* (por la FK).
- Si el auto está alquilado ($\text{disponibilidad} = \text{false}$) → **400 Bad Request**.

Capas a implementar:

- **AutoService.java** → eliminarAuto(Long id)
- **AutoServiceImpl.java** → validar disponibilidad y eliminar en orden correcto, con **@Transactional**
- **AutoRepository.java** y **Detalle_autoRepository.java**
- **AutoController.java** → endpoint DELETE

HU-22 Cancelar alquiler

Como cliente, quiero cancelar un alquiler antes de que inicie, para liberar la reserva.

Endpoint: DELETE /alquileres/{id}

Respuesta exitosa:

```
204 No Content
```

Reglas de negocio:

- Si no existe → **404 Not Found**.
- Si fecha_inicio ya pasó o es hoy → **400 Bad Request** con el mensaje *'El alquiler ya inició, no se puede cancelar'*.
- **Cambio v2:** si es válido, eliminar el alquiler y liberar el auto llamando a `autoService.actualizarDisponibilidad(idAuto, true)` (antes: `CatalogoFeignClient`).

Capas a implementar:

- **AlquilerService.java** → `cancelarAlquiler(Long id)`
- **AlquilerServiceImpl.java** → validar fecha, eliminar y liberar auto, con `@Transactional`
- `AlquilerRepository.java` → `findById()` y `deleteById()`
- `AlquilerController.java` → endpoint DELETE

HU-14 Registrar cliente

Como cliente, quiero registrarme en el sistema, para poder realizar alquileres.

Endpoint: POST /clientes

Body de entrada (JSON):

```
{ "nombre": "Juan Perez", "email": "juan@email.com",  
  "telefono": "3001234567", "tarjeta_credito": "4111111111111111" }
```

Respuesta exitosa:

```
{ "id_cliente": 1, "nombre": "Juan Perez", "email": "juan@email.com",  
  "telefono": "3001234567" }
```

Reglas de negocio:

- Todos los campos son obligatorios → **400** si falta alguno.
- Si el email ya está registrado → **400 Bad Request** con el mensaje 'El email ya existe'.
- **No retornar la tarjeta de crédito** en la respuesta.

Capas a implementar:

- Cliente.java → entidad JPA (tabla *clientes*)
- ClienteRepository.java → existsByEmail(String email)
- CreateClienteRequest.java / CreateClienteResponse.java (el response **no** lleva el campo tarjeta)
- ClienteMapper.java → garantiza que la tarjeta nunca salga en el DTO de respuesta
- **ClienteService.java** → crearCliente(CreateClienteRequest)
- **ClienteServiceImpl.java** → validación de email duplicado
- ClienteController.java → endpoint POST

HU-15 Actualizar datos de cliente

Como cliente, quiero actualizar mis datos de contacto, para mantener mi perfil al día.

Endpoint: PUT /clientes/{id}

Body de entrada (JSON):

```
{ "telefono": "3109876543", "tarjeta_credito": "4222222222222222" }
```

Respuesta exitosa:

```
{ "id_cliente": 1, "nombre": "Juan Perez", "email": "juan@email.com",  
  "telefono": "3109876543" }
```

Reglas de negocio:

- Si no existe → **404 Not Found**.
- No retornar la tarjeta en la respuesta.
- No permitir cambiar el email.

Capas a implementar:

- UpdateClienteRequest.java → sin campo email
- **ClienteService.java** → actualizarCliente(Long id, UpdateClienteRequest)
- **ClienteServiceImpl.java** → validar existencia y actualizar
- ClienteRepository.java → findById() y save()

- `ClienteController.java` → endpoint PUT

HU-16 Listar todos los clientes

Como administrador, quiero listar todos los clientes, para gestionar la base de usuarios.

Endpoint: GET `/clientes`

Respuesta exitosa:

```
[{ "id_cliente": 1, "nombre": "Juan Perez", "email": "juan@email.com",  
  "telefono": "3001234567", "tarjeta_credito": "*****1111" }]
```

Reglas de negocio:

- Enmascarar la tarjeta mostrando solo los últimos 4 dígitos.
- Si no hay clientes → retornar lista vacía.

Capas a implementar:

- **ClienteService.java** → `listarClientes()`
- **ClienteServiceImpl.java** → lógica de enmascarado de tarjeta
- **ClienteMapper.java** → método `enmascararTarjeta(String)`
- **ClienteRepository.java** → `findAll()`
- **ClienteController.java** → endpoint GET

HU-18 Crear alquiler

Como cliente, quiero crear un alquiler, para reservar el vehículo que necesito.

Endpoint: POST /alquileres

Body de entrada (JSON):

```
{ "id_cliente": 1, "id_auto": 1, "fecha_inicio": "2026-08-10",  
  "fecha_fin": "2026-08-13", "ciudad_retirada": "Bogota",  
  "ciudad_devolucion": "Medellin" }
```

Respuesta exitosa:

```
{ "id_alquiler": 1, "id_cliente": 1, "id_auto": 1, "fecha_inicio": "2026-08-10",  
  "fecha_fin": "2026-08-13", "precio_total": 405000,  
  "ciudad_retirada": "Bogota", "ciudad_devolucion": "Medellin" }
```

Reglas de negocio:

- **Cambio v2:** validar que el cliente exista con `clienteService.obtenerCliente(id)` → si no, **404** (antes: `ClienteFeignClient`).
- **Cambio v2:** validar que el auto esté disponible con `autoService.obtenerDetalle(id)` → si no, **400** (antes: `CatalogoFeignClient`).
- Calcular precio: $\text{dias} = \text{fecha_fin} - \text{fecha_inicio}$; $\text{precio_total} = \text{dias} * \text{precio_dia} * (1 - \text{oferta}/100)$.
- Marcar el auto como no disponible con `autoService.actualizarDisponibilidad(idAuto, false)`.
- `fecha_inicio` debe ser mayor a hoy → si no, **400**.

Capas a implementar:

- `Alquiler.java` → entidad JPA (tabla *alquileres*)
- `AlquilerRepository.java` → extiende `JpaRepository`
- `CreateAlquilerRequest.java` / `CreateAlquilerResponse.java`
- `AlquilerMapper.java` → entidad ↔ DTO
- **`AlquilerService.java`** → `crearAlquiler(CreateAlquilerRequest)`
- **`AlquilerServiceImpl.java`** → inyecta `ClienteService` y `AutoService`; lógica completa con validaciones y `@Transactional`
- `AlquilerController.java` → endpoint POST
- `ClienteFeignClient.java` y `CatalogoFeignClient.java` → **ya no se crean**

HU crítica del sprint — depende de otras

- Necesita mergeadas: **HU-14** (Cliente + `ClienteService`), **HU-08** (Auto + `AutoService`), **HU-11** (`actualizarDisponibilidad`) y **HU-12** (`obtenerDetalle`, para el precio con oferta). Coordinar el orden de merge con Murcia, Cifuentes, Suarez y Cardona.

HU-20 Historial de alquileres del cliente

Como cliente, quiero consultar mi historial de alquileres, para revisar mis reservas.

Endpoint: GET /alquileres?id_cliente=1

Respuesta exitosa:

```
[{ "id_alquiler": 1, "id_auto": 1, "fecha_inicio": "2026-08-10",  
  "fecha_fin": "2026-08-13", "precio_total": 405000,  
  "ciudad_retirada": "Bogota", "ciudad_devolucion": "Medellin" }]
```


Reglas de negocio:

- El parámetro `id_cliente` es obligatorio → si no se envía, **400**.
- Si el cliente no tiene alquileres → retornar lista vacía.

Capas a implementar:

- **AlquilerService.java** → `historialPorCliente(Long idCliente)`
- **AlquilerServiceImpl.java** → lógica de consulta
- `AlquilerRepository.java` → `findByIdCliente(Long idCliente)`
- `AlquilerController.java` → endpoint GET con `@RequestParam`

HU-21 Listar alquileres activos

Como administrador, quiero listar todos los alquileres activos, para tener visibilidad de la flota.

Endpoint: GET `/alquileres/activos`

Respuesta exitosa:

```
[{ "id_alquiler": 1, "id_cliente": 1, "id_auto": 1, "fecha_inicio": "2026-08-10",  
  "fecha_fin": "2026-08-13", "ciudad_retirada": "Bogota",  
  "precio_total": 405000 }]
```

Reglas de negocio:

- Un alquiler está activo si `fecha_fin` $\geq$ hoy y no fue cancelado.
- Si no hay activos → retornar lista vacía.

Capas a implementar:

- **AlquilerService.java** → `listarActivos()`
- **AlquilerServiceImpl.java** → filtrar por fecha y estado
- `AlquilerRepository.java` → `findByFechaFinGreaterThanOrEqualTo(LocalDate hoy)`
- `AlquilerController.java` → endpoint GET `/alquileres/activos`

Anexo

Convenciones comunes a todas las HU

Patrón de una HU de punta a punta

Toda HU recorre las mismas capas, en este orden. Ningún controller habla directamente con un repository.

```
Controller → Service (interfaz) → ServiceImpl → Repository → BD
    ↑               ↓
    DTO request/response    Mapper ↔ Model (entidad)
```

Por qué interfaz + implementación

- El controller depende de `TiendaService` (la abstracción), no de `TiendaServiceImpl`. Eso es el **principio D de SOLID** (inversión de dependencias), que es justo lo que el profesor va a buscar.
- Permite escribir tests con un *mock* del service sin levantar la base de datos.
- Si más adelante el proyecto vuelve a separarse en microservicios, la interfaz se mantiene y solo cambia la implementación.

Manejo de errores — una sola vez para todo el equipo

Situación	Excepción a lanzar	Código HTTP
El recurso no existe	<code>ResourceNotFoundException</code>	404 Not Found
Regla de negocio violada	<code>BadRequestException</code>	400 Bad Request
Campo obligatorio faltante	<code>@Valid + MethodArgumentNotValidException</code>	400 Bad Request

El `GlobalExceptionHandler` (anotado con `@RestControllerAdvice`) las traduce a la respuesta HTTP. Nadie devuelve `ResponseEntity.status(404)` a mano desde un controller.

Definición de Terminado (DoD)

- El endpoint responde correctamente probado en Postman (caso feliz + al menos un caso de error).
- Existen las clases de **las cinco capas**: controller, service (interfaz), serviceImpl, repository y DTOs.
- Los errores devuelven el código HTTP correcto vía excepciones, no `ResponseEntity` manual.
- El código compila (`mvn clean install`) sobre `develop` actualizado.
- PR abierto y revisado por al menos un compañero antes del merge.

Orden de merge sugerido para el sprint

- **Día 1:** Arango monta exception + HU-01 (entidad Tienda). Cifuentes crea entidades `Categoria`, `Auto` y `Detalle_auto` (HU-06, HU-08). Murcia crea entidad `Cliente` (HU-14). Pedroza crea entidad `Alquiler` (HU-18, sin lógica aún).
- **Día 2:** CRUDs simples que no dependen de nadie — HU-02, HU-03, HU-04, HU-05, HU-07, HU-15, HU-16, HU-09, HU-10.
- **Día 3:** HU-11 y HU-12 (las que consume Alquileres), luego HU-13.
- **Día 4:** HU-18 completa, y a continuación HU-20, HU-21, HU-22 y HU-24.